

คู่มือและสคริปต์สำหรับการนำเสนอ (Presentation Guide)

DEX L2E Platform - Secure API Management & Row-Level Security

ผมได้ทำการป้อนตารางข้อมูลจำลองสวยๆ และเชื่อมต่อ API แบบสมบูรณ์แบบลงในระบบของคุณแล้ว คุณสามารถเปิดหน้าเว็บ L2E ขึ้นมาแล้วนำเสนอตาม Step ด้านล่างนี้เพื่อให้ผู้บริหารหรือทีมงานเห็นภาพได้ทันทีครับ!

ไฮไลต์ที่จะนำเสนอ (Key Features)

Dynamic Data Provider: การแปลง Table ในฐานข้อมูลให้กลายเป็น API โดยไม่ต้องเขียนโค้ด

Row-Level Security (Scope): อาศัยสิทธิ์ API เดียวกัน ในการควบคุมว่า “ใครมองเห็นข้อมูลชุดไหน”

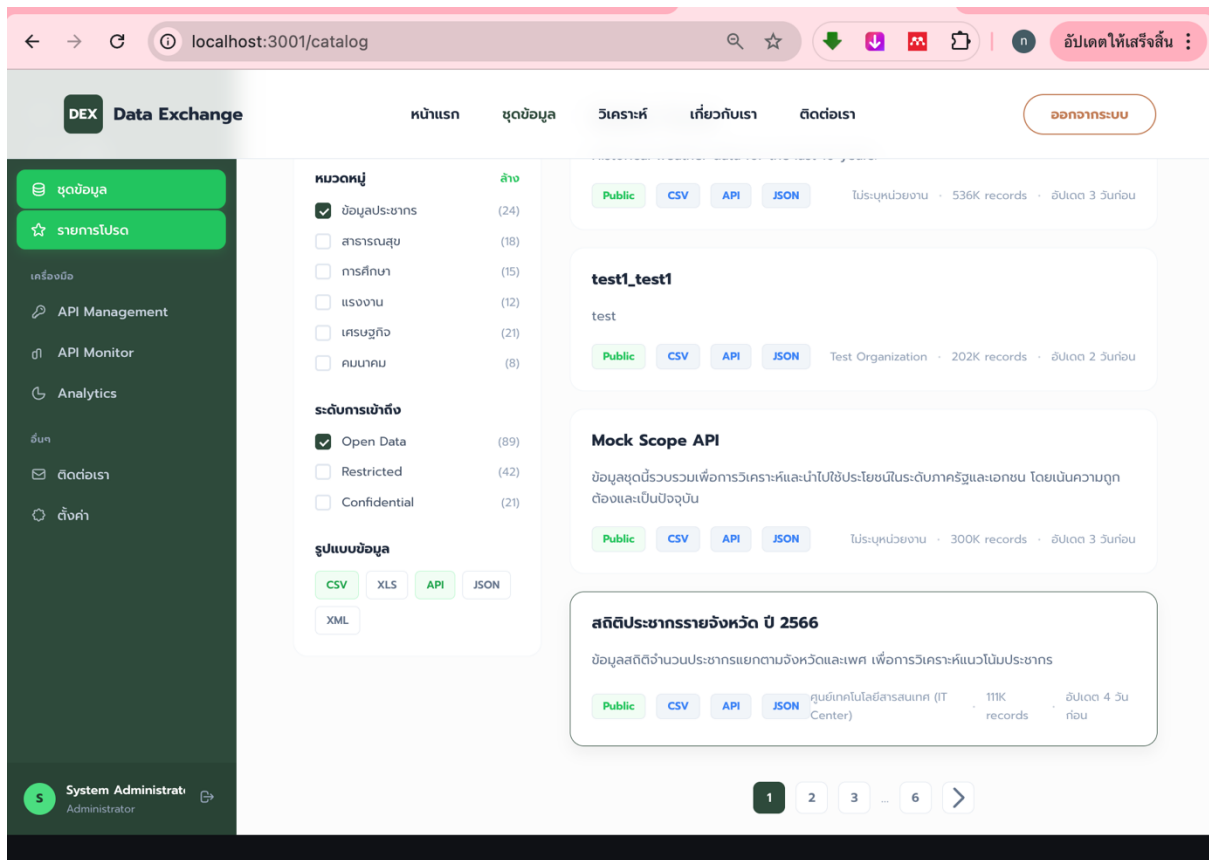
Usage Monitoring & Security: ควบคุมวันหมดอายุ, ระบุบัญชีฉุกเฉิน และบันทึกประวัติการถูกเรียกใช้

ขั้นตอนการกด Demo หน้าเว็บ (Demo Flow)

1. ผลลัพธ์ของข้อมูล (The "Mock_Population" Table)

คำพูดนำเสนอ: "ตอนนี้ในระบบ L2E ของเรามีข้อมูลตัวอย่างคือ 'สถิติประชากรรายจังหวัด ปี 2566' ซึ่งมีข้อมูลของทุกจังหวัดรวมอยู่ในหน้าตาเดียว"

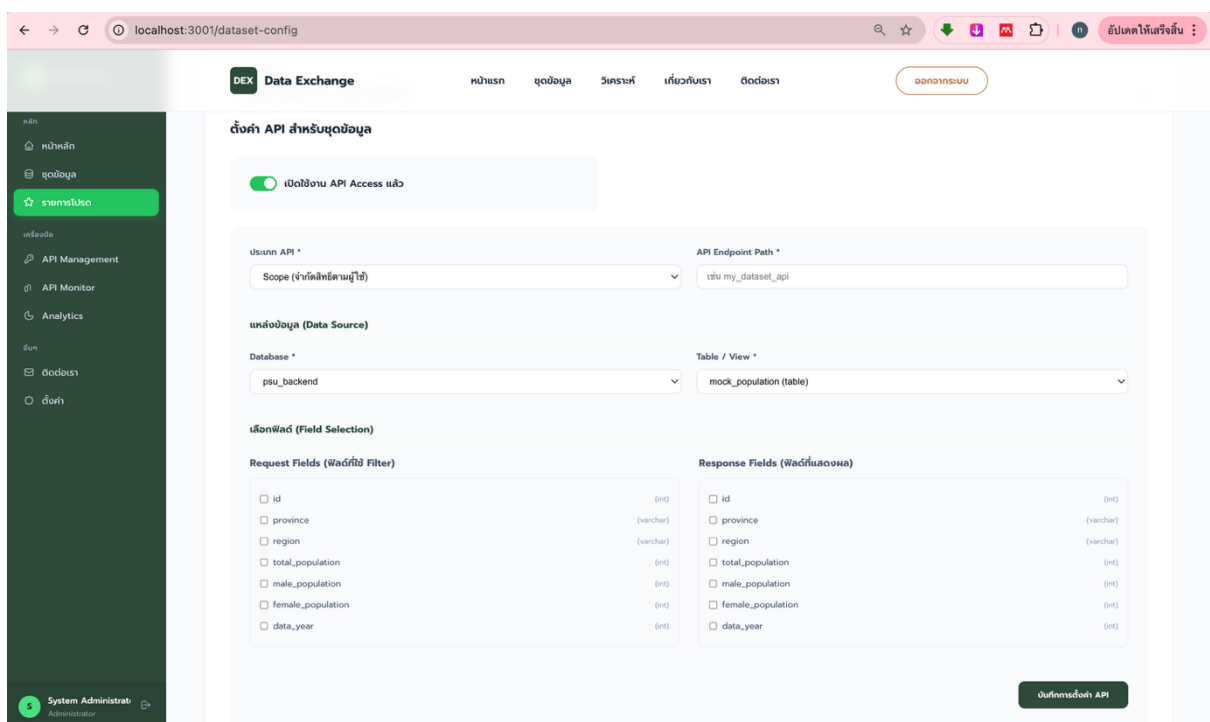
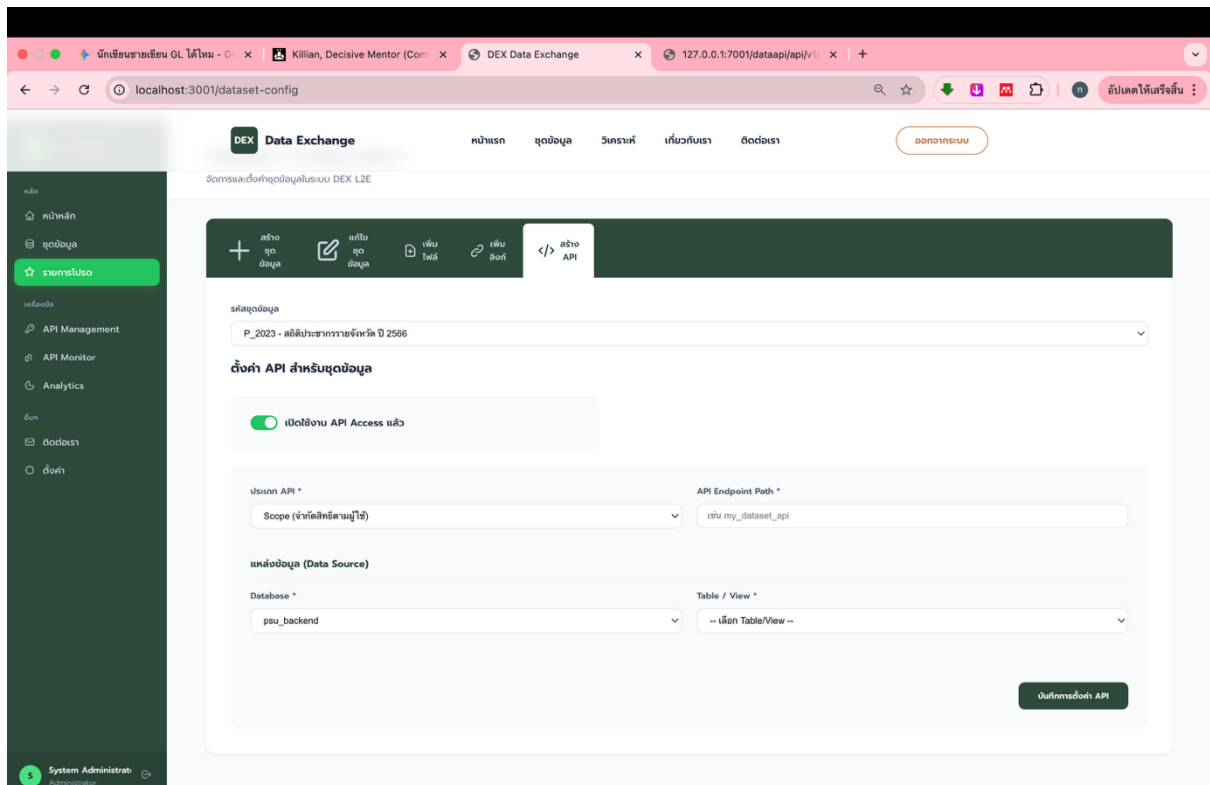
สิ่งที่เห็น: ในหน้าเว็บหมวดหมู่ Catalog คุณจะพบ Dataset ชื่อ สถิติประชากรรายจังหวัด ปี 2566 โผล่ขึ้นมาเป็นตัวล่าสุดครับ แนะนำเรื่องการแจกแจง Field ต่างๆ ได้เลย



2. การผูก API อัตโนมัติ (API Configuration)

ให้คลิกไปที่เมนู Dataset Configuration -> ค้นหา สถิติประชากรรายจังหวัด ปี 2566 -> ไปที่แท็บ สร้าง API

คำพูดนำเสนอ: "แอดมินไม่ต้องเขียนโค้ดเพิ่ม แค่กดเปิด 'api_enabled' แล้วเลือก Database psu_backend เชื่อมกับตาราง mock_population ระบบก็จะอ่าน Field ออกมาให้เลย แอดมินสามารถเลือกได้ว่า จะให้ User ส่งคำค้นหาด้วยฟิลด์ไหน (Request) และยอมให้ API ฟันข้อมูลฟิลด์ไหนกลับไป (Response)"



3. การจำกัดสิทธิ์ (API Management + Scope)

ให้คลิกไปที่เมนู API Management -> เลือก Service สถิติประชากรรายจังหวัด ปี 2566

คำพูดนำเสนอ: "เรามาดูการแจกกุญแจกันครับ สังเกตว่า API ชุดนี้มีป้าย (Badge) สีเหลืองแจ้งว่าเป็นแบบ Scope (Row-Level Security)"

หน้าจะแสดงข้อมูลจำลองที่ถูกเตรียมไว้:

นายสมชาย - กุญแจ DEMO_KEY_BKK_999 (โซว้การกำหนด Scope ให้เห็นแค่ province: ["กรุงเทพมหานคร"])

DEX Portal

หน้าหลัก

แดชบอร์ด

รายการโปรด

แดชบอร์ด

API Management

API Monitor

Analytics

ตั้งค่า

แจ้งเตือน

หน้าแรก

แดชบอร์ด

วิธการ

เกี่ยวกับเรา

ติดต่อเรา

ออกจากระบบ

API Management

จัดการ API Key และ Scope สำหรับผู้ใช้งาน

Service: คลังสินค้ารายสัปดาห์ ID 2566 (scope)

เพิ่ม API Key

Scope API (Row-Level Security)

API Credentials

รายการ API Key สำหรับ Service ที่เลือก

ผู้ใช้งาน	SECRET KEY	สถานะ	SCOPE	อายุการใช้งาน	จัดการ
bkk_admin สมชาย ใจดี	DEMO_KEY_BKK_999	Active	เลือก Scope	สร้าง: 2026-04-09 หมด: 2026-12-31	ลบข้อมูล แก้ไข
cnx_admin สมชาย ใจดี	*****	Active	เลือก Scope	สร้าง: 2026-04-09 หมด: 2026-12-31	ลบข้อมูล แก้ไข
temp_user สมชาย ใจดี	*****	Active	เพิ่ม Scope	สร้าง: 2026-04-09 หมด: 2023-01-01	ลบข้อมูล แก้ไข

Security Best Practices

ห้ามใช้ API Key

เก็บ Key ไว้เป็นความลับ หากถูก expose ให้แจ้งเตือนทันที

Scope = Row-Level Security

ใช้ Scope API เพื่อจำกัดข้อมูลตามผู้ใช้แต่ละคน

หมุนเวียน Key ทุก 90 วัน

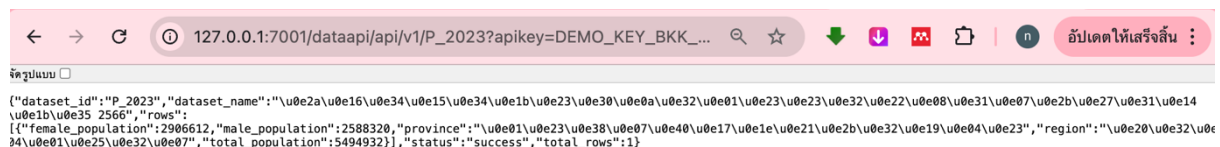
เปลี่ยน Key เป็นประจำเพื่อความปลอดภัยสูงสุด

4. ยิง API ให้ดูสดๆ (Live API Calling)

เปิดหน้า Postman หรือเบราว์เซอร์ แล้วยิง URL ตามนี้ให้ดูทีละเส้น:

🔥 ยิงด้วยคีย์ "คนกรุงเทพฯ" (DEMO_KEY_BKK_999) GET

http://127.0.0.1:7001/dataapi/api/v1/P_2023?apikey=DEMO_KEY_BKK_999 ผลลัพธ์ที่จะเกิด: ระบบจะฟันเฉพาะข้อมูลประชากรของ กรุงเทพมหานคร 1 เรคคอร์ดกลับมา (ปกป้องข้อมูลจังหวัดอื่นสมบูรณ์แบบ)



🔥 ยิงด้วยคีย์ "คนภาคเหนือ" (DEMO_KEY_CNX_555) GET

http://127.0.0.1:7001/dataapi/api/v1/P_2023?apikey=DEMO_KEY_CNX_555 ผลลัพธ์ที่จะเกิด: ระบบจะส่งข้อมูลกลับมาทั้งหมดที่อยู่ในภาคเหนือ (กรณีนี้คือ เชียงใหม่) เท่านั้น



🔥 ยิงด้วยคีย์ "ที่หมดอายุ" (DEMO_KEY_EXPIRED) GET

http://127.0.0.1:7001/dataapi/api/v1/P_2023?apikey=DEMO_KEY_EXPIRED ผลลัพธ์ที่จะเกิด: ระบบฟ้อง Error 403 Forbidden พร้อมข้อความ "API Key has expired" ตะกระเด็นทันที



5. ระบบความปลอดภัยย้อนหลัง (API Monitoring & Logging)

คำพูดนำเสนอ: "แล้วเรารู้ได้ยังไงว่ามีคนพยายามแฮ็กเอาคีย์หมดอายุมาใช้? ...ข้อมูลทั้งหมดถูกดูดลงถึง Log เรียบร้อยแล้วครับ"

ถ้าเข้าไปเช็คใน Database (ตาราง log โดย Query คำว่า `SELECT * FROM log WHERE type='API' ORDER BY log_id DESC;`) จะเห็นประวัติการยิงทั้งหมด รวมถึง IP และประเภท Error ถูกบันทึกไว้ทุกเลี้ยววินาที สร้างความมั่นใจต่อผู้บริหารถึงความโปร่งใสในข้อมูลได้เลยครับ!

คำแนะนำสำหรับการเปิดเบร่าว์เซอร์โชว์: ให้แน่ใจว่าเสียบสาย LAN / ใช้ IP ตรงกับเครื่อง Server ที่รัน (127.0.0.1 ถ้าอยู่เครื่องตัวเอง หรือ 110.X.X.X ถ้ารันแยกเครื่อง) และเปลี่ยนพอร์ตจาก 7001 เป็นพอร์ตจริงที่คุณใช้นะครับ

ไฟล์ `bigdataservice.py`

ขั้นที่ 1: รับรหัสและควานหาใน Database

เมื่อใครสักคนยิงโค้ดเข้ามา ระบบจะเอา API Key ของเขาไปเทียบกับฐานข้อมูลเพื่อดึง "วันหมดอายุและสถานะ" ออกมาก่อน

1. รับค่า apikey จาก URL ที่เขายิงเข้ามา

```
apikey = request.args.get('apikey')
```

2. ยิง SQL เข้าไปหาว่ากุญแจนี้มีในระบบไหม? สถานะอะไร? และวันหมดอายุคือตอนไหน?

```
sql_cred = """SELECT c.credential_id, c.service_id, c.user_id, c.status, s.api_enabled,  
s.service_id as s_id, c.expires_at
```

```
FROM api_credentials c
```

```
JOIN service s ON c.service_id = s.service_id
```

```
WHERE c.secret_key = %s AND s.dataset_id = %s"""
```

```
cursor.execute(sql_cred, (apikey, dataset_id))
```

```
cred_row = cursor.fetchone() # ดึงผลลัพธ์ใส่ตัวแปรแถว
```

ขั้นที่ 2: ประกาศตัวแปรเพื่อเตรียมเช็คบิล

บรรทัดนี้คือการ "แกะข้อมูล" ที่ได้จาก Database ออกมาใส่ตัวแปรทีละตัว เพื่อเตรียมเอาไปตั้งค่าตรวจสอบ

แยกคอลัมน์จาก DB ออกมาเป็นตัวแปร (เช่น cred_status คือสถานะ, expires_at คือวันหมดอายุ)

```
credential_id, svc_id_from_cred, user_id, cred_status, api_enabled, real_service_id,  
expires_at = cred_row
```

ขั้นที่ 3: ตรวจสอบที่ 1 - เช็คว่าโดนแอดมิน "ถอนออก (Revoke)" หรือไม่?

นี่คือจุดที่กันเรื่อง เอาโค้ดไปรันทิ้งไว้ที่บ้าน (Hardcode) ครบ ต่อให้รหัสถูกเป๊ะๆ แต่ถ้าแอดมินกดระงับ โค้ดตรงนี้จะเตะออกทันที

```
if cred_status != 'active': # ตรวจสอบว่าสถานะไม่ใช่ 'active' (อาจจะเป็น 'revoked')
```

```
# จด Log ประจານลง Database ทันทีว่าใครพยายามแอบเข้า
```

```
log_api_usage(user_id, 'API Key has been revoked', 403)
```

```
# ปิดการเชื่อมต่อ และเตะออกพร้อมพ่นข้อความ Error กลับไป
```

```
cursor.close()
```

```
conn.close()
```

```
return jsonify({'status': 'error', 'message': 'API Key has been revoked'}), 403
```

ขั้นที่ 4: ตรวจสอบที่ 2 - เช็คว่า "วันหมดอายุ" (Expiration)

นี่คือจุดที่ตัดขาดอายุการใช้งานครับ โดยใช้คำสั่งบรรทัดเดียวแต่ได้ผลดีมาก

```
# ถ้าคีย์นี้มีการตั้งวันหมดอายุเอาไว้ และวันหมดอายุนั้นน้อยกว่า "เวลาปัจจุบัน (datetime.now())"
```

```
if expires_at and expires_at < datetime.now():
```

```
    # จด Log ลง Database ทันทีว่าคีย์นี้หมดอายุไปแล้วแต่พยายามเรียกใช้
```

```
    log_api_usage(user_id, 'API Key has expired', 403)
```

```
    # เตะตัดขาดทันที ใช้งานไม่ได้เด็ดขาด
```

```
    cursor.close()
```

```
    conn.close()
```

```
    return jsonify({'status': 'error', 'message': 'API Key has expired'}), 403
```

สรุป: ด้วยโครงสร้างโค้ดการตั้งด่านที่ละสเตปแบบนี้ ทำให้เราไม่ต้องไปเขียนโค้ดอะไรไว้ที่เครื่องของ User เลยครับ แอดมินคุมอำนาจเปิดเสร็จผ่านตัวแปร status และ expires_at ในฐานข้อมูลได้ทั้งหมด